

LLM-Powered Knowledge Graph Extractor with Provenance, Event Reification, and Importance-Ranked Visualization

Mohamad Jehad Motaz Khachfa
800241913

CS585 Knowledge Engineering
mohamadjehad@aucegypt.edu

Abstract

Building useful knowledge bases from text has been a challenge for a long time. The standard process, known as the Knowledge Acquisition Bottleneck, requires a domain expert to read documents and write down entities and relations by hand. Large Language Models (LLMs) can now do most of this work in a single API call. This paper describes a web application that takes any text and returns an interactive knowledge graph. The system has four main design choices. First, it uses an open vocabulary for entity types with a controlled vocabulary of 23 relations. We started with 15 relations (the typical encyclopedic set) and added 8 more after we noticed that many biographical facts were collapsing to a generic `related_to` edge. Second, the prompt tells the model to use *event reification* for biographical events such as births and marriages, so that a person is never directly connected to a date. Third, the prompt asks the model to merge surface mentions of the same entity using an `aliases` field. Fourth, every edge carries a verbatim evidence sentence, a self-reported confidence in $[0, 1]$, and an explicit `[FALLBACK]` marker when the generic `related_to` predicate was used. The visualization shows confidence as a dashed line and fallbacks as a dotted italic line, so the user can see directly where the vocabulary fails. The frontend also includes an *importance-ranked complexity slider* that hides less central entities at lower levels, which is useful for paper-ready summary views. The system works with four LLM providers (Anthropic Claude, OpenAI GPT, Google Gemini, and local Ollama models) behind one shared schema, and exports the graph as JSON, PNG, or RDF/Turtle.

1 Introduction

Building queryable knowledge bases from text has historically been slow and expensive. Human experts need to read every document, decide which entities and relations are relevant, and write them down. Feigenbaum [1] called this the *Knowledge Acquisition Bottleneck*, and the problem has not really disappeared.

LLMs change this in a special way. They can recognize entities and relations in any new passage without being trained on a specific datasets, and they can be asked to return structured output (JSON with a fixed schema) using tool-use APIs [11] or strict JSON-Schema decoding [12]. This paper describes a web application that uses this capability to do most of what a knowledge engineer used to do by hand.

The main contributions are:

1. A hybrid extraction schema (Section 4) with open entity types and 23 controlled relations.
2. A decision-rule system prompt (Section 5) that tells the model which relation to use for each common pattern, and that asks it to do event reification, alias merging, and generic/specific deduplication during extraction.

3. A per-edge audit trail (Sections 4, 6): every edge has a verbatim evidence sentence, a confidence value in $[0, 1]$, and a `[FALLBACK]` prefix when the model used the generic relation.
4. An interactive visualization (Section 6) with degree-based node sizing, an ego-network highlight on hover, an importance-ranked complexity slider, and several filters that compose together.
5. A provider-agnostic backend (Section 3) covering Anthropic, OpenAI, Gemini, and Ollama behind the same schema, with RDF/Turtle export.

The rest of the paper is structured as follows. Section 2 reviews related work. Section 3 describes the architecture. Section 4 explains the knowledge model and Section 5 the prompt. Section 6 describes the visualization. Section 7 reports a qualitative evaluation. Section 8 discusses limitations. Section 9 concludes.

2 Related Work

Automated knowledge graph construction has gone through three main phases, and a few other research areas are also relevant for the choices we make below.

Rule-based and supervised pipelines. Early systems combined Named Entity Recognition with a separately trained Relation Extraction model. Mitchell et al.’s NELL system [2] showed that a semi-automated pipeline can keep building a knowledge base for years, but it still needed regular human curation, and it did not generalize well across domains. Adding a new relation meant collecting and labeling new training data.

Knowledge graph embeddings. A parallel line of work focused on learning vector representations for entities and relations. TransE [3] is the classic example: $\text{head} + \text{relation} \approx \text{tail}$ in the embedding space. Later models such as RotatE and ComplEx improved the geometry. These methods are mainly for reasoning over an existing graph (predicting missing links, finding inconsistencies). They are not directly useful for building a graph from raw text.

LLM-based extraction. Recent work uses LLMs as zero-shot or few-shot extractors. Pan et al. [4] survey this area and argue that LLMs and KGs should be combined: LLMs provide natural-language understanding, KGs provide structure and queryability. They specifically recommend controlled relations with open entity types, which is the choice we follow in Section 4. Microsoft’s GraphRAG [5] combines LLM extraction with graph-aware retrieval to improve question answering, and industry tools such as Neo4j’s LLM Graph Builder [6] package the same idea for users. Our system fits in this third phase, but we focus on three things that other tools handle less explicitly: vocabulary coverage as an auditable property, n-ary reification done inside the prompt, and an importance-based summary view.

Ontology design and n-ary relations. Noy and McGuinness’s *Ontology Development 101* [7] explains the standard way to handle relations that have more than two arguments: introduce an extra node for the event itself, and attach the arguments to that node. A marriage, for example, involves two people, a date, and sometimes a place. In our system, the prompt asks the LLM to do this reification at extraction time rather than as a post-processing step.

Entity resolution. Ji et al. [8] survey knowledge graph construction and completion. They list entity resolution (deciding when two surface forms refer to the same entity) as one of the recurring

failure modes of automated pipelines. The Maria Skłodowska / Marie Curie case from their survey is exactly the problem we address with the `aliases` field (Section 5.3).

Knowledge graph summarization. Visualization research has noticed for a long time that even small KGs become hard to read after a few dozen nodes. Liu et al. [9] review KG construction techniques and discuss summarization options based on structural features (degree, centrality) or semantic ones (query relevance, ontology depth). Our complexity slider (Section 6.4) is structural: it ranks entities by degree plus mention frequency, which is simple to compute and easy to explain.

Provenance. W3C’s PROV-O [10] and related work all stress that being able to trace a fact back to its source is necessary for trustworthy reuse. We follow a lightweight version of this: every node carries source sentences, every edge carries an evidence sentence and a confidence score, and the RDF export keeps these using standard `rdf:Statement` reification.

3 System Architecture

The project is a small web application with two parts: a backend that exposes the extraction API, and a frontend that handles the input, the visualization, and the export. They communicate over JSON HTTP. Figure 1 shows the high-level data flow.



Figure 1: High-level data flow. The frontend never talks to LLM providers directly. The backend selects the right provider and normalizes the result into a single graph schema. Only one LLM call is made per extraction; everything else (filtering, layout, complexity slider, export) happens in the browser.

The backend integrates with four LLM providers: Anthropic Claude, OpenAI GPT, Google Gemini, and local models served by Ollama. Each provider has a different way of returning structured output, but the backend hides these differences behind a single common interface. The user can switch between providers from a dropdown menu in the UI without any change to the application. A small registry detects automatically which providers are available (based on which API keys are configured) and shows only those in the menu.

4 Knowledge Model and Schema Design

The most important design choices in this kind of system are the knowledge model (what counts as an entity, what counts as a relation, what extra fields each one carries) and the prompt that asks for it.

4.1 Hybrid Schema: Open Types, Controlled Relations

A knowledge model can be *closed*, where both entity types and relation labels are fixed in advance, or *open*, where the extractor invents them. Closed models give clean, queryable graphs but only work in one domain. Open models handle any text but suffer from *label drift*: the same relation appears as *works_at*, *employed_by*, and *is_employee_of* in different runs, which breaks SPARQL queries and any union operation.

We use a *hybrid*: open entity types, but controlled relations. The model picks the most natural type for each entity (Person, Element, Theory, Field of study, etc.), while relation labels come from a fixed list. This split is also what Pan et al. [4] recommend.

4.2 Controlled Relation Vocabulary

The vocabulary contains 23 relations chosen to cover the patterns that appear most often in biographical, scientific, and organizational text. They fall into three rough categories:

- *Biographical place relations.* A person is *born in* a city, or *lived in* a country. Using a single `located_in` for both organizations and people would be a type error, so we keep them separated.
- *Discovery vs. creation.* Natural elements, particles, and laws are *discovered*, not *created*. We provide `discovered_by` alongside `created_by` and `developed_by` so that elements like polonium are not assigned a human-authorship relation.
- *Shared honours.* When a Nobel Prize is shared between three people, the symmetric co-recipient relation needs to survive as more than a generic connection.

The full list is:

```
works_for, member_of, founded_by, located_in, headquartered_in, part_of,  
created_by, developed_by, discovered_by, causes, influences, occurred_at,  
occurred_on, has_property, related_to, opposes, born_in, lived_in, married_to,  
studied_at, awarded, awarded_for, co_recipient_with.
```

A `related_to` relation is included as a deliberate fallback: when no other relation in the list fits a fact in the text, the model uses this one and prefixes the evidence string with [FALLBACK] (see Section 5.5). The visualization shows fallback edges in a distinct style, which makes vocabulary gaps visible and easy to audit for any new domain.

4.3 Output Schema

The extraction target is a JSON object with two arrays. Listing 1 shows the schema in abbreviated form.

Listing 1: .]Output schema (abbreviated). Each node carries its source sentences and any aliases. Each edge carries the verbatim evidence sentence and a confidence value in [0,1].

```
{  
  "nodes": [{  
    "id": "marie_curie",  
    "label": "Marie Curie",  
    "type": "Person",  
    "description": "Polish-born French physicist and chemist...",  
    "source_sentences": ["Marie Curie was a Polish-born physicist..."],  
    "aliases": ["Maria Sklodowska"]  
  }],  
  "edges": [{  
    "source": "polonium",  
    "target": "marie_curie",  
    "relation": "discovered_by",  
    "evidence": "...they discovered the elements polonium and radium.",  
    "confidence": 0.95  
  }]  
}
```

Three things to note. First, every single node has source sentences and every edge has an evidence sentence. This lets the user check the extraction, and it also makes the model less likely to invent facts: when you ask the model to cite a sentence, it tends to stay closer to the text. Second, the `id` field uses `lowercase_with_underscores`, which works directly as a URI fragment in the RDF/Turtle export (`kg:marie_curie`). Third, `aliases` and `confidence` are required (not optional) in the schema, because OpenAI's `strict:true` mode does not allow optional properties; the model returns `[]` or a default of 0.7 when there is nothing to say.

5 The Prompt

Beyond listing the relation vocabulary, the prompt does five things, each one designed to solve a specific extraction problem.

5.1 Decision Rules for Each Relation

To prevent over-use of the generic `related_to` fallback, the prompt includes short decision rules for the more easily confused relations. For example:

```
A PERSON born in a place → born_in (NEVER located_in).  
A NATURAL phenomenon or element identified by a scientist → discovered_by, not  
created_by.  
Two or more people sharing an award → co_recipient_with between each pair.
```

5.2 Event Reification

Biographical events such as births and marriages usually have more than two arguments (a person, a date, sometimes a place). Since each edge is binary, attaching them directly to one person produces type errors like `marie_curie occurred_on 1867`, which reads as “Marie Curie happened on 1867”.

The standard fix from Noy and McGuinness [7] is to introduce an intermediate node for the event and attach the arguments to it (e.g. `birth_marie_curie occurred_on 1867`, `birth_marie_curie occurred_at warsaw`). The prompt asks the model to perform this reification at extraction time. The cost is more nodes; we offset this in the visualization by rendering event nodes smaller and hiding them first when the complexity slider is moved down (Section 6.4).

5.3 Entity Resolution via Aliases

When the same entity has multiple surface names (“Maria Skłodowska” / “Marie Curie”), a naive extractor produces disconnected nodes. The prompt instructs the model to emit a single node with the canonical label and list alternatives under an `aliases` field. In the side panel, aliases appear as small italic pills under the main label.

5.4 Deduplicating Generic and Specific Mentions

When a class and its instances both appear (“the Nobel Prize” alongside “the Nobel Prize in Physics”), the prompt asks the model to prefer the specific instances and link them with `part_of` to the class node only if the class is worth keeping on its own.

5.5 Confidence and the [FALLBACK] Marker

Each edge carries a self-reported `confidence` value in `[0, 1]`: 1.0 for relations stated with an unambiguous verb, 0.7 for relations clearly implied, 0.4 for relations inferred from context. Edges with confidence below 0.6 render as dashed lines.

In parallel, when the model uses the generic `related_to` relation, the evidence string must start with the token `[FALLBACK]`. These edges render as dotted italic muted lines with a `fallback`

badge in the side panel. A graph dominated by such edges signals that the current vocabulary does not fit the input domain.

6 Interactive Visualization

The frontend is a single page that puts the text input and provider menus on the left and the graph view on the right. The graph is drawn with Cytoscape.js, with a toolbar above it and a side panel that opens when a node is clicked.

6.1 Visual Encoding

Node color encodes the entity type (Person, Organization, Location, Concept, Event, plus a default). Node size is proportional to degree, so the most central entities visually stand out. Edge style encodes confidence: solid for high-confidence vocabulary edges, dashed for low confidence (< 0.6), and dotted-italic-muted for [FALLBACK] edges. Event-reification nodes are rendered smaller and semi-transparent, so they fade behind the real entities. Edge labels are hidden by default and revealed only on hover, otherwise the graph becomes unreadable when there are many edges.

6.2 Layouts

Five layouts are available: force-directed, concentric (rings by degree), hierarchy, circle, and grid. The default is chosen automatically: concentric if one node has at least 40% of the possible degree share (a typical hub), otherwise force-directed.

6.3 Filters

The toolbar has four filters that can be combined freely:

- **Relation chips:** hide all edges of a chosen relation.
- **Per-entity hide:** a “hide” button in the side panel removes a specific entity; hidden entities reappear from the toolbar with one click.
- **Dates toggle:** hides nodes whose label looks like a year, useful for clean screenshots.
- **Complexity slider:** described next.

6.4 Importance-Ranked Complexity Slider

The slider trims the graph down to its most important entities. Each entity gets a score equal to its degree plus twice the number of source sentences in which it is mentioned:

$$\text{score}(n) = \text{deg}(n) + 2 \cdot |\text{source_sentences}(n)|.$$

The mention term makes real entities score higher than event-reification nodes (which usually have no source sentences), so reified events are hidden first when the slider is moved down. The number of levels scales with graph size (3 for tiny graphs, up to 7 for large ones), and at each level only the top-ranked entities and the edges between them are shown. The slider composes with the manual hide list via a simple set union, so a peripheral entity hidden at maximum complexity stays hidden when the slider is dropped.

6.5 Ego Network and the Side Panel

Hovering a node fades the rest of the graph and highlights its closed neighborhood. Clicking a node opens the side panel, which shows the entity’s label, any aliases, type, description, source sentences, and the lists of incoming and outgoing relations. Each relation row has a confidence badge and, for [FALLBACK] edges, a `fallback` marker.

6.6 Export

Three buttons in the graph header: JSON (the normalized graph), RDF/Turtle (with reified evidence and confidence), and PNG (a $2\times$ -resolution snapshot, used for the figures in this paper).

7 Qualitative Evaluation

We evaluated the system on three passages of different character: a simple organizational sentence, a longer abstract passage from a research document, and a biographical paragraph that uses an alias. The outputs were inspected manually for entity coverage, relation correctness, alias resolution, and fallback rate. Table 1 summarizes the size of each extracted graph.

Table 1: Extraction results on three sample passages (Claude Sonnet 4.5).

Passage	Nodes	Edges	[FALLBACK]	Aliases
Microsoft	6	5	0	—
Research Aims	~ 35	~ 50	several	—
Mark Twain	14	17	1	1 (Clemens)

7.1 Sample 1: Microsoft

“Microsoft is headquartered in Redmond, Washington. Bill Gates founded the company, which develops Windows and Office.”

The extraction is clean and contains no fallbacks: `microsoft headquartered_in redmond, redmond located_in washington, microsoft founded_by bill_gates, windows developed_by microsoft, office developed_by microsoft`. Microsoft is connected to four of the five other entities (hub ratio = 0.8), so the page automatically picks the concentric layout. The result confirms that the controlled vocabulary covers simple organizational text without any fallback.

7.2 Sample 2: Research Aims

The second passage is the “Specific Research Aims” chapter from a thesis proposal, roughly 200 words describing four research aims for a tracing SDK project. It is the most challenging input we tested. Many of its verbs (*build, measure, quantify, validate*) do not map cleanly to the controlled vocabulary, so the model uses `related_to` several times. The full graph contains about 35 nodes and 50 edges, which is too dense to read at once.

This is exactly the case the importance-ranked complexity slider was designed for. At level 1, only the two most central aims and their immediate neighbours are shown, giving a clean elevator-pitch view. At level 2, all four aims become visible together with their connecting concepts. At the maximum level, the full graph is exposed for detailed inspection. The slider therefore turns a single extraction into a continuous summary–detail axis. The frequent dotted [FALLBACK] edges also signal correctly that the vocabulary, which is tuned for biographical and organizational text, does not fully cover an abstract research-process domain.

7.3 Sample 3: Mark Twain

“Mark Twain, born Samuel Clemens in Missouri, wrote The Adventures of Tom Sawyer and Adventures of Huckleberry Finn. He married Olivia Langdon in 1870, and the couple settled in Connecticut. Twain received an honorary doctorate from Oxford University in 1907.”

The model emits a single `mark_twain` node with `Samuel Clemens` stored as an alias, which is the expected outcome of the alias-resolution rule in the prompt. The biographical relations fire as designed: `married_to` (Olivia Langdon, symmetric in both directions), `created_by`

for the two novels, and `awarded` for the honorary doctorate. Birth, marriage, settlement, and award are reified as their own event nodes, each carrying `occurred_on` (date) and `occurred_at` (place) attributes, which avoids the type errors of attaching dates directly to Twain. One `[FALLBACK]` edge connects the award event to `Oxford University` — the vocabulary has no precise `awarded_by` relation, and the audit marker correctly highlights that gap.

7.4 Cross-Provider Notes

Two observations recurred across all three samples. First, schema-enforced providers (Anthropic tool-use, OpenAI strict JSON schema) produced parseable output every time, while Ollama with `format: "json"` occasionally returned relations outside the controlled vocabulary, which the normalizer coerced to `related_to`. Second, Claude tends to extract more entities and reify more aggressively, while Gemini is faster and produces a coarser graph; the complexity slider lets the user normalize Claude’s output toward Gemini’s level if they prefer a cleaner summary.

8 Discussion and Limitations

The system handles short, well-formed English passages well, but several limitations remain. The vocabulary is fixed at 23 relations, so specialized domains (legal, biomedical, financial) would need additional ones; the `[FALLBACK]` marker is what tells the user when the current vocabulary stops being enough. Confidence values are self-reported by the LLM and not calibrated against any external scorer, so they should be read as hints rather than measurements. Each extraction is single-pass with no cross-document entity linking, so submitting the same text twice produces two separate nodes. Finally, the evaluation is qualitative and based on three passages: a quantitative study against gold-standard annotations remains future work.

9 Conclusion

We have built an LLM-powered knowledge graph extractor that takes any short text and produces an interactive graph in seconds. The extraction layer uses a hybrid model (open entity types, 23 controlled relations), in-prompt event reification, alias-based entity merging, generic/specific deduplication, self-reported confidence, and an explicit `[FALLBACK]` marker. The interaction layer uses degree-based node sizing, an ego-network highlight on hover, five layouts with a smart default, four orthogonal filters, an importance-ranked complexity slider, and JSON / RDF / PNG export.

The main lesson from the project is that classical knowledge-engineering problems do not disappear when you use an LLM. The vocabulary still drifts, n-ary relations still need reification, the same entity still appears under multiple names, and a graph that does not show its uncertainty is still hard to trust. What changes is who does the work: a single LLM call now does most of what a human knowledge engineer used to do by hand, and the engineer’s job becomes designing the rules (the vocabulary, the prompt, the audit) that make the extraction trustworthy.

Cross-document entity linking against an external knowledge base, a quantitative evaluation against gold-standard annotations, and a user-editable relation vocabulary are the three most natural next steps. The codebase is small enough that each of these can be added without major changes to the architecture.

References

- [1] Feigenbaum, E.A.: The art of artificial intelligence: themes and case studies of knowledge engineering. In: Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1014–1029 (1977).

- [2] Mitchell, T., Cohen, W., Hruschka, E., Talukdar, P., Yang, B., Betteridge, J., Carlson, A., Dalvi, B., Gardner, M., Kisiel, B., et al.: Never-Ending Learning. *Communications of the ACM* **61**(5), 103–115 (2018).
- [3] Bordes, A., Usunier, N., García-Durán, A., Weston, J., Yakhnenko, O.: Translating Embeddings for Modeling Multi-relational Data. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2013).
- [4] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X.: Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024). <https://arxiv.org/abs/2306.08302>
- [5] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *Microsoft Research* (2024). <https://arxiv.org/abs/2404.16130>
- [6] Neo4j: LLM Graph Builder: Transforming Unstructured Data into Knowledge Graphs (2024). <https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/>
- [7] Noy, N.F., McGuinness, D.L.: *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05 (2001).
- [8] Ji, S., Pan, S., Cambria, E., Marttinen, P., Yu, P.S.: A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. *IEEE Transactions on Neural Networks and Learning Systems* **33**(2), 494–514 (2022).
- [9] Liu, Q., Li, Y., Duan, H., Liu, Y., Qin, Z.: Knowledge Graph Construction Techniques. *Journal of Computer Research and Development* **53**(3), 582–600 (2016).
- [10] Lebo, T., Sahoo, S., McGuinness, D., et al.: PROV-O: The PROV Ontology. W3C Recommendation (2013). <https://www.w3.org/TR/prov-o/>
- [11] Anthropic: Tool use with Claude. Anthropic API documentation (2024). <https://docs.anthropic.com/claude/docs/tool-use>
- [12] OpenAI: Structured Outputs. OpenAI API documentation (2024). <https://platform.openai.com/docs/guides/structured-outputs>